知识图谱

1.1 概述

1.1.1 RAG局限

(1) 多跳推理（Multi-hop Reasoning）

问题："张三的导师的学生中，谁在2023年获得了国家自然科学基金？"

这个问题需要三步推理：

1. 找到张三的导师是谁
2. 找到该导师的所有学生
3. 在这些学生中筛选2023年获国自然基金的人

(2) 结构化关系查询（Structured Relational Query）

问题："列出所有总部在深圳、员工超过5万人、且在2022年进行过IPO的科技公司。"

这是一个典型的多条件过滤+聚合查询，本质上是一个SQL语句：

```sql
SELECT name FROM companies
WHERE headquarters = '深圳'
  AND employees > 50000
  AND ipo_year = 2022
  AND sector = '科技';
```

(3) 事实一致性（Fact Consistency）

问题："阿里巴巴的创始人是谁？"

语料中可能存在以下段落：

- 段落A："阿里巴巴集团由马云于1999年在杭州创立。"
- 段落B："阿里巴巴的创始人包括马云、蔡崇信等18位合伙人。"

- 段落C（过时文档）："阿里巴巴CEO马云表示……"（暗示马云是CEO，但实际他已于2019年卸任）

1.1.2 定义

知识图谱（Knowledge Graph） 是一种用图结构来表示知识的技术，它以**实体（Entity）**为节点，以**关系（Relation）**为边，将现实世界中的事物及其关联组织成一个结构化的语义网络。

最简单的知识图谱就是一组**三元组（Triple）**：

$(\text{头实体}, \text{关系}, \text{尾实体})$　即　(h, r, t)

例如：`（高血压，导致，头痛）`、`（氨氯地平，治疗，高血压）`、`（阿司匹林，禁忌，胃溃疡）`。

这些三元组组合在一起，就形成了一个网状的知识结构

1.1.3 与 RAG 结合

目前工业界有三种主流架构：

架构一：KG作为RAG的"索引增强层"

在这种架构中，知识图谱不直接回答问题，而是用来**增强RAG的检索精度。**

优势：实现简单，不需要改变RAG的主流程
局限：知识图谱只用于"扩展检索词"，没有发挥其推理能力

架构二：KG作为独立的"结构化问答通道"

在这种架构中，系统有两个并行的问答通道：

- **非结构化通道**：标准RAG，处理开放式、叙述性问题
- **结构化通道**：知识图谱问答（KGQA），处理结构化、多跳问题

系统首先对用户问题做**意图分类**（Intent Classification），判断该走哪个通道，然后分别处理，最后合并答案。

优势：每个通道处理自己擅长的问题，精度高
局限：意图分类本身可能出错；两个通道的答案可能不一致

架构三：GraphRAG——深度融合

这是目前最前沿的架构，我们将在第四篇详细展开。核心思想是：

- 用知识图谱的**社区检测**（Community Detection）对文档进行语义聚类
- 为每个社区生成**摘要**，实现"全局理解"
- 查询时根据问题类型路由到"局部检索"（传统RAG）或"全局摘要"（图谱社区）
- 利用图结构做**多跳推理**，把推理路径作为上下文喂给LLM

1.2 图论

没有图论，只能"用"知识图谱（写Cypher查询）；有了图论，你才能"理解"知识图谱（为什么某些查询慢、如何设计推理算法、如何做链接预测）。

具体来说，图论在以下场景中是必须的：

- **查询优化**：判断一个图查询的时间复杂度，需要图论中的遍历算法分析
- **知识推理**：基于路径的推理（PRA）本质上是图上的随机游走，需要图论中的路径和连通性概念
- **图神经网络**：GNN的消息传递机制建立在图的邻接矩阵之上，需要理解图的矩阵表示
- **社区检测**：GraphRAG中的社区检测需要图论中的聚类和子图概念

1.2.1 图的基本定义

定义（有向图）：

一个有向图（Directed Graph）G 是一个二元组 $G = (V, E)$，其中：

- $V = \{v_1, v_2, \ldots, v_N\}$ 是顶点（Vertex/Node）的有限集合，$N = |V|$ 是顶点数
- $E \subseteq V \times V$ 是有向边（Directed Edge）的有限集合，每条边 $e = (u, v) \in E$ 表示从 u 指向 v 的有向连接

定义（加权有向图）：一个加权有向图是三元组 $G = (V, E, W)$，其中 $W : E \to \mathbb{R}$ 是边的权重函数。

知识图谱与有向图的对应关系：

- 知识图谱的实体 $\mathcal{E}$ ↔ 图的顶点 V
- 知识图谱的三元组 (h, r, t) ↔ 一条带标签的有向边 $h \to t$，标签为 r
- 知识图谱是**多关系有向图**（Multi-relational Graph）：两个顶点之间可以有多条不同类型的边

1.2.2 度（Degree）

定义（出度与入度）：在有向图中，顶点 v 的出度（Out-degree）和入度（In-degree）分别定义为：

$$d_{\text{out}}(v) = |\{(v, u) \in E \mid u \in V\}|$$

$$d_{\text{in}}(v) = |\{(u, v) \in E \mid u \in V\}|$$

顶点 v 的总度（Total Degree）：$d(v) = d_{\text{out}}(v) + d_{\text{in}}(v)$。

在知识图谱中的意义：

- **入度高的实体**是"热门实体"。很多其他实体指向它。例如"中国"作为"国籍"关系的尾实体，入度极高
- **出度高的实体**是信息丰富的实体"。比如 "阿里巴巴"，它有总部、CEO、成立时间、经营范围..... 出度很高
- **度分布**通常服从幂律分布（Power Law）：少数实体拥有大量连接，大多数实体连接很少。

1.2.3 路径（Path）与连通性（Connectivity）

定义（路径）：在有向图中，一条从 v_0 到 v_k 的长度为 k 的路径是一个顶点序列 $P = (v_0, v_1, \ldots, v_k)$，满足对所有 $0 \leq i < k$，有 $(v_i, v_{i+1}) \in E$。

- **简单路径**（Simple Path）：路径中所有顶点互不相同
- **环**（Cycle）：$v_0 = v_k$ 且其余顶点互不相同的路径

定义（可达性）：如果存在从 u 到 v 的路径，则称 v 从 u 可达（Reachable），记作 $u \rightsquigarrow v$。

定义（强连通分量）：有向图的一个强连通分量（Strongly Connected Component, SCC）是一个最大顶点子集 $C \subseteq V$，满足对任意 $u, v \in C$，都有 $u \rightsquigarrow v$ 且 $v \rightsquigarrow u$。

在知识图谱中的意义：

- **路径就是推理链**：从"张三"到"杭州"的路径 `张三 →(工作于)→ 阿里巴巴 →(总部位于)→ 杭州`，就是一条两跳推理链
- **连通性分析**可以发现知识图谱中的"孤岛"——与主图不连通的实体集合，通常意味着抽取遗漏或数据质量问题
- **最短路径**可以用来衡量两个实体之间的"语义距离"，在实体对齐和推荐中有用

1.2.4 子图（Subgraph）

定义（子图）：图 $G' = (V', E')$ 是图 $G = (V, E)$ 的子图，当且仅当 $V' \subseteq V$ 且 $E' \subseteq E \cap (V' \times V')$。

定义（导出子图）：给定顶点子集 $S \subseteq V$，由 S 导出的子图（Induced Subgraph）$G[S] = (S, E_S)$，其中 $E_S = \{(u, v) \in E \mid u \in S, v \in S\}$。

在知识图谱中的意义：

- **子图匹配**是图查询的基础：用户的查询本质上是一个"查询图"，在知识图谱中找到所有匹配的子图
- **社区检测**就是找到"内部连接密集、外部连接稀疏"的子图
- **GraphRAG中的"局部上下文"**就是以查询实体为中心的一个子图（通常是k-hop邻域）

1.2.5 图的邻接矩阵与路径计数

给定多关系图 $G = (V, E, R)$，关系集合 $R = \{r_1, \ldots, r_K\}$。对每个关系 r_k，邻接矩阵 $\mathbf{A}^{(k)} \in \{0, 1\}^{N \times N}$。

定理（k跳路径计数）：从顶点 v_i 出发，依次经过关系类型 $r_{a_1}, r_{a_2}, \ldots, r_{a_m}$，到达顶点 v_j 的路径数量为：

$$P_{ij}^{(a_1,\ldots,a_m)} = \left(\mathbf{A}^{(a_1)} \cdot \mathbf{A}^{(a_2)} \cdots \mathbf{A}^{(a_m)} \right)_{ij}$$

- **基例**（$m = 1$）：$P_{ij}^{(a_1)} = \mathbf{A}_{ij}^{(a_1)}$，即直接边的存在性，显然成立。
- **归纳步**：假设 $m = k$ 时成立，即 $P_{ij}^{(a_1,\ldots,a_k)} = \left(\prod_{t=1}^{k} \mathbf{A}^{(a_t)} \right)_{ij}$ 表示从 i 经过 k 步到达 j 的路径数。

  对于 $m = k+1$，从 i 经过 $k+1$ 步到达 j 的路径，可以分解为：从 i 经过 k 步到达某个中间顶点 l，再从 l 经过一步 $r_{a_{k+1}}$ 到达 j。对所有可能的中间顶点 l 求和：

  $$P_{ij}^{(a_1,\ldots,a_{k+1})} = \sum_{l=1}^{N} P_{il}^{(a_1,\ldots,a_k)} \cdot \mathbf{A}_{lj}^{(a_{k+1})}$$

  根据矩阵乘法的定义，这正是 $\left(\prod_{t=1}^{k} \mathbf{A}^{(a_t)} \right) \cdot \mathbf{A}^{(a_{k+1})}$ 的第 (i,j) 个元素。

  因此 $m = k+1$ 时也成立。

1.3 数据模型

知识图谱有两种主流的数据模型：**RDF（资源描述框架）**和**属性图（Property Graph）**。

- **RDF** 诞生于W3C语义网社区，设计目标是"数据的互联网化"——让数据像网页一样可以互相链接、被机器理解。它强调标准化、互操作性和逻辑推理。
- **属性图** 诞生于图数据库工业界（Neo4j是代表），设计目标是"高效的图存储和查询"。它强调工程实用性、查询性能和开发体验。

1.3.1 RDF 模型

RDF三元组：RDF的基本数据单元是三元组 (s, p, o)，即（主语subject，谓语predicate，宾语object）。

- **主语（s）**：一个IRI（Internationalized Resource Identifier，国际化资源标识符），代表被描述的资源
- **谓语（p）**：一个IRI，代表主语和宾语之间的关系
- **宾语（o）**：一个IRI或一个字面量（Literal，如字符串、数字、日期）

RDF 的特点是**标准化、语义化**，适合数据交换和推理，但不支持在关系上挂属性

- **全局唯一标识**：每个实体和关系都有IRI，确保跨数据集的互操作性
- **统一的三元组结构**：属性和关系统一用三元组表示，没有"节点属性"的概念
- **标准化的推理语言**：支持RDFS（RDF Schema）和OWL（Web Ontology Language）做逻辑推理
- **标准化的查询语言**：SPARQL

1.3.2 属性图模型

属性图（Property Graph）是一个有向图，其中：

- **节点（Node/Vertex）**：每个节点有一个唯一ID，一个或多个标签（Label，代表节点类型），以及一组属性（键值对）
- **边（Edge/Relationship）**：每条边有一个唯一ID，一个类型（Type），一个起始节点和一个终止节点，以及一组属性

属性图（Property Graph）是 Neo4j 等图数据库采用的模型，它的特点是**节点和关系都可以带属性**，更灵活、更适合工程实践。

1.4 Schema与Ontology

Schema的本质作用：Schema（模式）或Ontology（本体）是知识图谱的"宪法"，它定义了：

- 有哪些**实体类型**（如Person、Company、Product）
- 有哪些**关系类型**（如founder、headquarter、brand）
- 每种关系的**定义域**（Domain）和**值域**（Range），即什么类型的实体可以作为头/尾

- 关系的**约束**（如对称性、传递性、函数性）
- 实体类型之间的**层次关系**（如Student是Person的子类）

Ontology的核心组件

一个Ontology $\mathcal{O}$ 通常包含以下组件：

1. **类（Class/Concept）**：实体类型的集合，记为 $\mathcal{C}$。类之间有层次关系（子类继承）。
   - 形式化：$C_1 \sqsubseteq C_2$ 表示 C_1 是 C_2 的子类（所有 C_1 的实例都是 C_2 的实例）
2. **属性（Property/Role）**：关系类型的集合，记为 $\mathcal{P}$。分为：
   - **对象属性**（Object Property）：连接两个实体，$P : C_1 \times C_2$
   - **数据属性**（Datatype Property）：连接实体和字面量，$P : C \times \mathcal{D}$（$\mathcal{D}$ 是数据类型域）
3. **属性约束**：
   - **定义域**（Domain）：$\exists P.\, C$ 表示 P 的定义域是 C
   - **值域**（Range）：$C \exists P$ 表示 P 的值域是 C
   - **函数性**（Functional）：如果 P 是函数性的，则每个实体最多有一个 P 关系（如"出生日期"）
   - **逆函数性**（Inverse Functional）
   - **对称性**（Symmetric）：$P(x, y) \Rightarrow P(y, x)$（如"配偶"）
   - **传递性**（Transitive）：$P(x, y) \wedge P(y, z) \Rightarrow P(x, z)$（如"祖先"）
4. **公理（Axiom）**：逻辑规则，用于推理和约束检查

1.5 实体、关系、属性、类型

在知识图谱的实践中，"实体""关系""属性""类型"这四个概念经常被混用。比如：

- "红色"是实体还是属性值？
- "朋友"是关系类型还是实体类型？
- "张三是一个学生"中的"学生"是类型还是属性？

混淆的根源在于：这些概念的边界不是绝对的，而是**取决于建模目的和粒度**。但理解它们的"标准定义"和"常见辨析"，能帮你在90%的场景下做出正确选择。

概念	定义	图中的表示	例子
实体（Entity）	现实世界中可区分、可识别的具体事物或抽象概念	节点（Node）	张三、阿里巴巴、杭州、物理学
关系（Relation）	两个实体之间的语义关联	边（Edge），有方向和类型	创始人、总部位于、出生于
属性（Attribute）	实体或关系的内在特征，值为字面量	节点或边的键值对	姓名、年龄、成立年份、任职时间
类型（Type/Class）	实体的分类，代表一组具有共同特征的实体	节点的标签（Label）或`rdf:type`关系	Person、Company、City

1.6 知识表示学习

将每个实体和关系表示为一个低维稠密向量（通常50-300维），使得向量空间中的几何关系能够编码知识图谱中的语义关系。

这与NLP中的Word Embedding（如Word2Vec）思想一致，"你可以通过一个词周围的词来理解它"（You shall know a word by the company it keeps）。在知识图谱中，"你可以通过一个实体参与的三元组来理解它"。

阶段一：One-Hot表示

最简单的表示方法：每个实体对应一个维度。

$$\mathbf{e}_i = [0, \ldots, 0, \underbrace{1}_{\text{第}i\text{位}}, 0, \ldots, 0] \in \{0,1\}^N$$

其中 N 是实体总数。

阶段二：基于共现的分布式表示

借鉴Word2Vec的思想，用实体在三元组中的"共现"来学习向量。但简单的共现忽略了**关系类型**的信息——`(张三，工作于，阿里巴巴)`和`(张三，创始人，阿里巴巴)`中的"张三"和"阿里巴巴"的关系是完全不同的。

阶段三：基于平移的知识图谱嵌入（TransE系列）

这是当前的主流方法。核心思想是：**把关系看作实体之间的"平移向量"**。如果 (h, r, t) 是一个正确的三元组，那么头实体的向量加上关系向量应该接近尾实体的向量：

$$\mathbf{h} + \mathbf{r} \approx \mathbf{t}$$

这个简单的想法，开启了知识表示学习的整个领域。

1.6.1 TransE

(1) 背景

在TransE之前（2013年），知识图谱嵌入方法主要是基于张量分解的（如RESCAL）。

这些方法把知识图谱表示为一个三维张量（实体×实体×关系），然后做张量分解。

问题：

- 参数数量大：RESCAL的参数量是 $O(N^2K)$（N实体数，K关系数），对于大规模图谱不可行
- 模型复杂，训练慢
- 效果并不理想

(2) 核心

关系是实体空间中的平移向量。 如果 (h, r, t) 成立，那么 $\mathbf{h} + \mathbf{r}$ 应该接近 $\mathbf{t}$。

类比：如果"北京"的向量是 $(1, 0)$，"中国"的向量是 $(2, 0)$，那么"首都"这个关系的向量就应该是 $(1, 0)$——因为"北京 + 首都 = 中国"。

(3) 公式: 评分函数与损失函数

评分函数：对于一个三元组 (h, r, t)，TransE定义其"可信度分数"为：

$$f_r(h, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_{L_1/L_2}$$

分数越高（距离越小），三元组越可能是正确的。

目标函数：我们希望正确三元组的分数高，错误三元组的分数低。使用**基于间隔的排序损失**（Margin-based Ranking Loss）：

$$\mathcal{L} = \sum_{(h,r,t)\in\mathcal{T}} \sum_{(h',r,t')\in\mathcal{T}'} \max\left(0, \gamma + f_r(h', t') - f_r(h, t)\right)$$

其中：

- $\gamma > 0$ 是间隔（margin）超参数
- $\mathcal{T}'$ 是负样本三元组集合
- $\max(0, x)$ 是Hinge Loss

推导直觉：

- 对于正确三元组 (h, r, t)，$f_r(h, t)$ 应该高（距离小）
- 对于错误三元组 (h', r, t')，$f_r(h', t')$ 应该低（距离大）
- 我们希望 $f_r(h, t) - f_r(h', t') > \gamma$，即正确和错误之间至少有 γ 的间隔
- 当这个条件不满足时（$f_r(h, t) - f_r(h', t') \leq \gamma$），损失为 $\gamma + f_r(h', t') - f_r(h, t) > 0$，需要优化

负采样策略：如何生成负样本 $\mathcal{T}'$?

TransE使用**随机替换**策略：对于正确三元组 (h, r, t)，随机替换头实体或尾实体（但不同时替换）：

$$\mathcal{T}' = \{(h', r, t) \mid h' \in \mathcal{E}\} \cup \{(h, r, t') \mid t' \in \mathcal{E}\}$$

(4) TransE缺陷

TransE虽然简单有效，但存在三个不可回避的缺陷：

缺陷一：无法处理复杂关系（1对N、N对1、N对N）

考虑关系"创始人"：一个公司可能有多个创始人（1对N）。TransE要求 $\mathbf{h} + \mathbf{r} = \mathbf{t}$，这意味着对于同一个公司 h 和关系 r，所有创始人 $t_1, t_2, \ldots$ 的向量都必须相同——这显然不合理。

数学上，如果 (h, r, t_1) 和 (h, r, t_2) 都成立，TransE要求：

$$\mathbf{h} + \mathbf{r} = \mathbf{t}_1 \quad \text{且} \quad \mathbf{h} + \mathbf{r} = \mathbf{t}_2$$

这推出 $\mathbf{t}_1 = \mathbf{t}_2$，即所有尾实体向量相同。这是TransE的根本矛盾。

缺陷二：无法处理多关系语义

同一个实体在不同关系中应该有不同的"侧面"。比如"马云"在"创始人"关系中是"企业家"侧面，在"出生于"关系中是"个人"侧面。TransE中每个实体只有一个向量，无法表达这种多侧面性。

缺陷三：无法建模对称关系

对称关系如"配偶"：如果 $(A, \text{配偶}, B)$ 成立，则 $(B, \text{配偶}, A)$ 也成立。TransE要求 $\mathbf{A} + \mathbf{r} = \mathbf{B}$ 且 $\mathbf{B} + \mathbf{r} = \mathbf{A}$，这推出 $\mathbf{r} = 0$，即关系向量为零向量——失去了区分能力。

1.6.2 ComplEx：复数空间嵌入

(1) 问题背景 如何建模对称和反对称关系?

TransE系列模型的一个共同问题是无法区分对称关系和反对称关系：

- **对称关系**：$(h, r, t) \Rightarrow (t, r, h)$，如"配偶""同学"
- **反对称关系**：$(h, r, t) \Rightarrow \neg(t, r, h)$，如"父亲""大于"

在实数空间中，如果 $\mathbf{h} + \mathbf{r} = \mathbf{t}$，那么 $\mathbf{t} + \mathbf{r} = \mathbf{h}$ 要求 $\mathbf{r} = -\mathbf{r}$，即 $\mathbf{r} = 0$。这意味着对称关系的关系向量必须为零，失去了区分能力。

ComplEx（Complex Embedding，Trouillon等人，2016）的思路是：**在复数空间中建模，利用复数的共轭性质来区分对称和反对称关系。**

(2) 核心

在复数空间中，一个向量 $\mathbf{z} = \mathbf{a} + i\mathbf{b}$，其共轭为 $\bar{\mathbf{z}} = \mathbf{a} - i\mathbf{b}$。

如果我们定义评分函数涉及共轭运算，那么：

- 对于对称关系，$f(h,t) = f(t,h)$ 可以自然成立
- 对于反对称关系，$f(h,t) = -f(t,h)$ 也可以自然成立

(3) 评分函数使用双线性形式（不是平移形式）：

$$f_r(h,t) = \mathrm{Re}\left(\langle \mathbf{h}, \mathbf{r}, \bar{\mathbf{t}} \rangle\right) = \mathrm{Re}\left(\sum_{i=1}^{d} h_i r_i \bar{t}_i\right)$$

其中 $\mathrm{Re}(\cdot)$ 表示取实部，$\bar{\mathbf{t}}$ 是 $\mathbf{t}$ 的共轭。

为什么能建模对称/反对称关系?

展开评分函数。设 $\mathbf{h} = \mathbf{h}_a + i\mathbf{h}_b$，$\mathbf{r} = \mathbf{r}_a + i\mathbf{r}_b$，$\mathbf{t} = \mathbf{t}_a + i\mathbf{t}_b$，其中下标 a 表示实部，b 表示虚部。

$$h_i r_i \bar{t}_i = (h_{a,i} + ih_{b,i})(r_{a,i} + ir_{b,i})(t_{a,i} - it_{b,i})$$

先计算前两项：

$$(h_{a,i} + ih_{b,i})(r_{a,i} + ir_{b,i}) = (h_{a,i}r_{a,i} - h_{b,i}r_{b,i}) + i(h_{a,i}r_{b,i} + h_{b,i}r_{a,i})$$

再乘以 $\bar{t}_i = t_{a,i} - it_{b,i}$：

$$= [(h_{a,i}r_{a,i} - h_{b,i}r_{b,i})t_{a,i} + (h_{a,i}r_{b,i} + h_{b,i}r_{a,i})t_{b,i}] + i[\ldots]$$

取实部：

$$\mathrm{Re}(h_i r_i \bar{t}_i) = h_{a,i}r_{a,i}t_{a,i} - h_{b,i}r_{b,i}t_{a,i} + h_{a,i}r_{b,i}t_{b,i} + h_{b,i}r_{a,i}t_{b,i}$$

现在看 $f_r(t,h)$（交换h和t）：

$$\mathrm{Re}(t_i r_i \bar{h}_i) = t_{a,i}r_{a,i}h_{a,i} - t_{b,i}r_{b,i}h_{a,i} + t_{a,i}r_{b,i}h_{b,i} + t_{b,i}r_{a,i}h_{b,i}$$

损失函数

ComplEx使用**对数似然损失**（而非Margin Loss）：

$$\mathcal{L} = -\frac{1}{|\mathcal{T}|} \sum_{(h,r,t)\in\mathcal{T}} \log(\sigma(f_r(h,t))) - \frac{1}{|\mathcal{T}'|} \sum_{(h',r,t')\in\mathcal{T}'} \log(1-\sigma(f_r(h',t'))$$

其中 $\sigma(\cdot)$ 是sigmoid函数。这本质上是一个二分类损失：正确三元组标签为1，负样本标签为0。

1.7 知识图谱构建

现实世界中，90%以上的知识以非结构化文本（网页、论文、报告、病历）的形式存在。要把这些文本变成结构化的知识图谱，需要经过四个核心环节：

- **知识抽取**（Knowledge Extraction）：从文本中识别实体、关系、属性、事件
- **知识融合**（Knowledge Fusion）：把不同来源的同一实体对齐，消解冲突
- **知识推理与补全**（Knowledge Reasoning & Completion）：基于已有知识推断缺失的三元组
- **知识存储与查询**（Knowledge Storage & Query）：把图谱存进图数据库，支持高效查询

1.7.1 知识抽取（Knowledge Extraction）

假设你有这样一段文本：

> "阿里巴巴集团由马云于1999年在杭州创立，旗下拥有淘宝、天猫等电商平台。2014年，阿里巴巴在纽约证券交易所上市，融资额达250亿美元。"

人类很容易从中提取出以下三元组：

- `(阿里巴巴，创始人，马云)`
- `(阿里巴巴，成立年份，1999)`
- `(阿里巴巴，总部位于，杭州)`
- `(阿里巴巴，旗下拥有，淘宝)`
- `(阿里巴巴，旗下拥有，天猫)`

- `(阿里巴巴，上市地点，纽约证券交易所)`
- `(阿里巴巴，上市年份，2014)`
- `(阿里巴巴，融资额，250亿美元)`

但让机器做这件事，面临三个核心困难：

困难一：实体边界的模糊性

"阿里巴巴集团"和"阿里巴巴"是同一个实体吗？"纽约证券交易所"应该作为一个整体实体，还是"纽约"+"证券交易所"两个实体？

困难二：关系的隐含性

"由马云于1999年在杭州创立"，一句话中包含了创始人、成立年份、成立地点三个关系，需要同时抽取。

困难三：歧义性

"马云"可能指阿里巴巴创始人，也可能指其他同名的人。"苹果"可能指水果，也可能指公司。

(2) 实体识别（NER）：从CRF到大模型

NER的四代方法

实体识别（Named Entity Recognition, NER）的目标是从文本中识别出实体的**边界**和**类型**。

第一代：规则与词典

方法：人工编写正则表达式规则（如"大写字母开头的连续词"）和实体词典（如公司名词典、人名词典），对文本做匹配。

问题：

- 规则无法覆盖所有语言现象（如"阿里"这种简称、"阿里巴巴集团"这种全称）
- 词典无法覆盖新兴实体（如新成立的公司、新出现的网红）

第二代：CRF（条件随机场）

HMM的局限

在CRF之前，序列标注用的是HMM（隐马尔可夫模型）。HMM做了两个强假设：

- **齐次马尔可夫假设**：当前标签只依赖前一个标签
- **观测独立性假设**：当前观测只依赖当前标签

这两个假设在NER中都不成立：

- 实体标签不仅依赖前一个标签，还可能依赖更前面的标签（如"B-ORG"后面不一定是"I-ORG"，也可能是"O"）
- 当前词的标签不仅依赖当前词，还依赖上下文（如"苹果"在"苹果手机"中是公司，在"苹果水果"中是水果）

CRF（Conditional Random Field）放弃了HMM的观测独立性假设，直接建模**给定观测序列条件下标签序列的条件概率** $P(\mathbf{y}|\mathbf{x})$，而不是联合概率 $P(\mathbf{x}, \mathbf{y})$。

CRF的局限

- 依赖人工特征工程（词形、词性、词典特征等）
- 无法建模长距离依赖（虽然比HMM好，但本质上还是局部的）
- 特征设计需要领域专家，迁移到新领域成本高

第三代：BiLSTM-CRF

CRF的特征工程瓶颈

CRF的效果高度依赖特征工程的质量。为了让CRF识别出"阿里巴巴"是组织，需要人工设计"是否在公司名词典中""是否以'公司''集团'结尾"等特征。这在新领域中需要重新设计。

BiLSTM-CRF的核心改进是：**用双向LSTM自动从文本中学习特征，替代人工特征工程，然后接CRF层做序列解码。**

架构

```
输入句子 → 词嵌入(Word Embedding) → 双向LSTM → 发射分数 → CRF层 → 标签序列
```

- **BiLSTM**：自动提取上下文特征，输出每个位置的发射分数（每个标签的分数）
- **CRF层**：考虑标签之间的转移依赖，做全局最优解码

BiLSTM-CRF的局限：

- LSTM对长距离依赖的建模仍然有限（虽然比CRF好）
- 词嵌入是从头训练的，没有利用大规模预训练知识
- 在低资源场景下效果不佳

第四代：BERT-NER与大模型Prompt抽取

预训练语言模型的革命

2018年BERT的出现彻底改变了NLP。

BERT通过在大规模无标注文本上做掩码语言模型（MLM）预训练，学习到了丰富的上下文表示。在NER任务上，只需在BERT之上接一个简单的分类层（或CRF层），微调就能达到SOTA效果。

BERT-NER架构

```
输入句子 → [CLS] token1 token2 ... [SEP] → BERT编码器 → 每个token的隐状态 → 线性层/CRF → 标签
```

与BiLSTM-CRF的区别：

- 用BERT替代BiLSTM做特征提取
- BERT的自注意力机制天然支持长距离依赖
- 预训练知识大幅提升了低资源场景下的效果

【大模型Prompt抽取】

随着GPT-3、LLaMA等大模型的出现，NER可以通过**Prompt工程**实现，无需微调：

```
Prompt:
请从以下文本中提取所有的人名和组织名，以JSON格式输出。
文本：阿里巴巴集团由马云于1999年在杭州创立。

输出：
{"人名": ["马云"], "组织名": ["阿里巴巴集团"]}
```

优势

- 零样本/少样本能力强，不需要标注数据
- 可以灵活定义实体类型（只需修改Prompt）
- 能处理嵌套实体和非连续实体（传统序列标注难以处理）

局限

- 推理成本高（大模型推理慢且贵）
- 输出格式不稳定（可能不遵循JSON格式）
- 长文本处理困难（受上下文窗口限制）
- 实体边界可能不够精确

1.7.2 关系抽取（Relation Extraction）

NER解决了"文本中有哪些实体"的问题，但知识图谱需要的是三元组 (h, r, t)，实体之间的关系。

关系抽取的目标是：给定文本和一对实体 (h, t)，判断它们之间的关系 r 是什么。

例如，给定句子"阿里巴巴由马云创立"和实体对(阿里巴巴, 马云)，关系抽取模型需要输出"创始人"。

(1) 流水线 vs 联合抽取

关系抽取有两种范式：

流水线（Pipeline）：先做NER识别实体，再对每对实体做关系分类。

- 优点：模块解耦，每个模块可以独立优化

- 缺点：误差传播（NER错了，关系抽取必然错）；实体对组合爆炸（N个实体有$O(N^2)$对）

联合抽取（Joint Extraction）：同时识别实体和关系。

- 优点：避免误差传播，实体和关系互相约束
- 缺点：模型更复杂，训练更难

(2) 基于远程监督的关系抽取

关系抽取需要大量标注数据（句子+实体对+关系标签），人工标注成本极高。

远程监督（Distant Supervision）的核心思想是：如果知识库中存在三元组(h, r, t)，那么所有同时包含 h 和 t 的句子都被认为表达了关系 r。

例如，知识库中有`(阿里巴巴，创始人，马云)`，那么所有同时包含"阿里巴巴"和"马云"的句子都被标注为"创始人"关系的训练样本。

(3) 基于大模型的关系抽取

当前最主流的方法是用大模型做关系抽取，有两种范式：

范式一：分类式Prompt

```
请判断以下句子中，实体A和实体B之间的关系属于以下哪一类：
[创始人，总部位于，旗下拥有，上市地点，无关系]

句子：阿里巴巴集团由马云于1999年在杭州创立。
实体A：阿里巴巴
实体B：马云
关系：_____
```

范式二：生成式Prompt（端到端三元组抽取）

```
请从以下文本中提取所有的三元组（头实体，关系，尾实体），以JSON列表格式输出。
关系类型包括：创始人、总部位于、旗下拥有、成立年份、上市地点。

文本：阿里巴巴集团由马云于1999年在杭州创立，旗下拥有淘宝、天猫等电商平台。

输出：
[
  {"头实体": "阿里巴巴", "关系": "创始人", "尾实体": "马云"},
  {"头实体": "阿里巴巴", "关系": "成立年份", "尾实体": "1999"},
  {"头实体": "阿里巴巴", "关系": "总部位于", "尾实体": "杭州"},
  {"头实体": "阿里巴巴", "关系": "旗下拥有", "尾实体": "淘宝"},
  {"头实体": "阿里巴巴", "关系": "旗下拥有", "尾实体": "天猫"}
]
```

生成式Prompt的优势是端到端——不需要先做NER再做关系分类，一次输出所有三元组。但需要处理输出格式的稳定性问题。

(4)【代码实现】基于大模型的端到端三元组抽取

```python
import json
import re
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

def extract_triples(text,relation_types):
    llm = ChatOpenAI(
        model="deepseek-chat",
        api_key=os.getenv("DEEPSEEK_API_KEY"),
        base_url=os.getenv("DEEPSEEK_BASE_URL"),
        temperature=0,  # 低温度保证输出稳定、格式一致
    )
```

```
    prompt = f"""你是一个专业的知识图谱构建助手。请从以下文本中提取所有的三元组（头实体，关系，尾实体）。
关系类型限定为：{', '.join(relation_types)}
如果文本中没有这些关系，请返回空列表。
要求：
1. 实体必须是文本中出现的具体名称，不要泛化
2. 关系必须从给定的关系类型中选择
3. 输出严格的JSON数组格式，不要有其他文字说明
4. 每个三元组格式：{{"head": "头实体", "relation": "关系", "tail": "尾实体"}}
文本：
{text}
输出："""

    # 调用模型
    response = llm.invoke([{"role": "user", "content": prompt}])
    result = response.content.strip()

    # 解析JSON（处理模型可能输出的 markdown 代码块包裹）
    result = re.sub(r'^```json\s*', '', result)
    result = re.sub(r'\s*```$', '', result)

    try:
        triples = json.loads(result)
        return triples
    except json.JSONDecodeError:
        print(f"JSON解析失败，原始输出：{result}")
        return []


text = "阿里巴巴集团由马云于1999年在杭州创立，旗下拥有淘宝、天猫等电商平台。2014年，阿里巴巴在纽约证券交易所上市。"
relation_types = ["创始人", "成立年份", "总部位于", "旗下拥有", "上市地点", "上市年份"]

triples = extract_triples(text, relation_types)
print(json.dumps(triples, ensure_ascii=False, indent=2))
```

1.7.3 属性抽取与事件抽取

属性抽取（Attribute Extraction）

属性抽取的目标是从文本中提取实体的属性值（键值对）。例如从"马云，1964年9月10日出生于浙江杭州"中提取`(马云，出生日期，1964-09-10)`和`(马云，出生地，杭州)`。

属性抽取与关系抽取的区别：

- 关系抽取的尾实体是另一个实体（如"马云"）
- 属性抽取的尾值是字面量（如日期、数字、字符串）

方法：

- 基于规则：正则表达式（如日期、电话号码、邮箱）
- 基于模型：序列标注（BIO标注属性值）+ 属性分类
- 基于大模型：Prompt直接提取键值对

事件抽取（Event Extraction）

事件抽取的目标是从文本中识别事件的**触发词**（Trigger）和**论元**（Argument）。

例如，从"阿里巴巴2014年在纽交所上市"中：

- 事件类型：上市（IPO）
- 触发词：上市
- 论元：公司=阿里巴巴，时间=2014年，地点=纽交所

1.8 知识融合（Knowledge Fusion）

把不同来源、不同表述的同一实体和关系对齐、合并、消解冲突，形成一个统一、干净的知识图谱。

知识融合包含三个子任务：

1. **实体对齐**（Entity Alignment）：识别不同名称指向的同一实体

2. **本体对齐**（Ontology Alignment）：识别不同Schema中等价或相似的类和关系
3. **冲突检测与消解**（Conflict Detection & Resolution）：当不同来源对同一事实有矛盾时，决定相信谁

(1) 实体对齐（Entity Alignment)

同一实体为什么会有不同表述?

实体名称不一致的原因有四类：

原因	示例	难度
全称/简称	阿里巴巴集团 vs 阿里	低
中英文翻译	阿里巴巴 vs Alibaba Group	中
拼写/格式差异	Jack Ma vs 马云 vs 马云（带空格）	低
完全不同的名称	苹果公司 vs Apple Inc. vs 苹果电脑公司	高
同名异义	苹果（水果）vs 苹果（公司）	高（需要消歧）

实体对齐的本质

实体对齐的本质是：判断两个实体描述是否指向同一个现实世界对象。

这与RAG中的"语义相似度"有相似之处，但关键区别在于：实体对齐不仅要看名称的字面相似度，还要看实体的属性和关系结构，两个名称不同但属性高度一致的实体（如相同的成立年份、相同的总部地址），很可能是同一个实体。

方法一：基于属性相似度的对齐

比较两个实体的属性值，计算综合相似度。

给定两个实体 e_1 和 e_2，它们的属性集合分别为 $A_1 = \{(k_1, v_1), \ldots\}$ 和 $A_2 = \{(k'_1, v'_1), \ldots\}$。

方法二：基于嵌入的对齐

用知识图谱嵌入（如TransE）把实体映射到向量空间，语义相同的实体会在空间中靠近。

但不同知识图谱的嵌入空间是独立的，不能直接比较。需要跨图谱嵌入对齐。

方法三：基于大模型的对齐

核心思想：把两个实体的名称、属性、上下文描述喂给大模型，让大模型判断是否为同一实体。

```
Prompt:
请判断以下两个实体是否指向同一个现实世界对象。
实体A：名称="阿里巴巴集团"，属性={成立年份:1999，总部:杭州，创始人:马云}
实体B：名称="Alibaba Group"，属性={foundedYear:1999, headquarters:Hangzhou, founder:Jack Ma}

请回答"是"或"否"，并简要说明理由。
```

(2) 本体对齐（Ontology Alignment）

假设你要合并两个电商知识图谱：

- 图谱A的关系类型：`品牌`、`生产商`、`制造商`
- 图谱B的关系类型：`brand`、`manufacturer`、`producer`

这些关系类型语义上有重叠但不完全等价。本体对齐的目标是识别这些等价或相似关系，并建立映射。

(3) 冲突检测与消解

冲突的典型场景：

- 来源A：`(阿里巴巴，CEO，张勇)`（2020年数据）
- 来源B：`(阿里巴巴，CEO，吴泳铭)`（2023年数据）
- 来源C：`(阿里巴巴，员工数，100000)`
- 来源D：`(阿里巴巴，员工数，250000)`

策略	方法	适用场景
时间戳优先	保留最新的数据	有时效性的属性（CEO、员工数）
来源可信度加权	高可信度来源优先	来源质量差异大时
投票法	多数来源一致的值	多个独立来源时
保留多值+标注	不强行消解，保留所有值并标注来源和时间	不确定哪个对时
人工审核	交专家判断	关键事实、高风险场景

1.9 知识推理与补全

即使你完成了抽取和融合，得到的知识图谱仍然是**不完整的**。原因有三：

- **抽取遗漏**：文本中提到但模型没抽出来的关系
- **知识本身的隐含性**：有些知识不需要文本明确说明，可以通过推理得到。例如已知`(张三，父亲，张大山)`和`(张大山，父亲，张爷爷)`，可以推理出`(张三，祖父，张爷爷)`
- **数据源的局限性**：没有任何数据源能覆盖所有知识

知识推理与补全的目标是：基于图谱中已有的知识，推断出缺失的三元组。

(1) 基于规则的推理（Symbolic Reasoning）

最朴素的推理方式是**写规则**。如果你知道"如果A是B的父亲，B是C的父亲，那么A是C的祖父"，就可以用这条规则从已有三元组推导出新三元组。

示例：

```
祖父(?x，?z) ← 父亲(?x，?y)，父亲(?y，?z)
```

表示：如果x是y的父亲，y是z的父亲，那么x是z的祖父。

基于规则推理的三个致命缺陷

- **规则获取困难**：谁来写规则？人工写规则成本极高，且无法覆盖所有推理模式

- **脆弱性**：规则是精确匹配，数据有一点噪声（如关系名拼写错误）就无法触发
- **无法处理不确定性**：现实世界的知识很多是概率性的（"吸烟的人大概率得肺癌"），规则只能处理确定性知识

(2) 基于路径的推理（Path-based Reasoning)

如果规则写不全，能不能让算法自动从图中发现推理路径?

例如，算法发现图中很多"祖父"关系都对应着"父亲→父亲"的两跳路径，就可以自动学到这条推理规则。

PRA（Path Ranking Algorithm)

把关系推理建模为路径分类问题。对于目标关系 r，找到从 h 到 t 的所有路径，用路径特征训练一个分类器，判断 (h, r, t) 是否成立。

路径特征：一条路径是一个关系类型序列，如 `父亲→父亲`、`工作于→总部位于`。

算法流程：

1. 对每个目标关系 r，收集正样本（已知的 (h, r, t)）和负样本
2. 对每个样本，用随机游走找到从 h 到 t 的所有路径（限制长度）
3. 统计每种路径类型的出现频率，作为特征向量
4. 训练逻辑回归分类器
5. 推理时，对候选 (h, t) 提取路径特征，用分类器预测

(3) 神经符号融合推理（Neural-Symbolic Reasoning)

用神经网络处理不确定性和泛化，用符号推理保证可解释性和精确性。

当前最实用的路线是大模型+图数据库：

1. 大模型理解自然语言问题，生成推理计划（如"先查张三的父亲，再查父亲的父亲"）
2. 图数据库执行精确的图查询，返回结构化结果
3. 大模型把结果组织成自然语言回答

这本质上是"大模型做语义理解，图数据库做结构化推理"的神经符号融合。第四篇讲GraphRAG时会详细展开。